

Laboratorio #1 — Reporte Grupal

Esquemas de comunicación

1. Integrantes

Pareja

- Javier López – 23415
- Javier Cifuentes - 23079

Pareja colaboradora (parte 3.3)

- Javier Benitez
- Olivier Viau

2. Introducción

En esta primera parte del laboratorio transmitimos mensajes usando código Morse y código de Baudot con la estación de telégrafo. Primero lo hicimos en tiempo real, luego mediante notas de voz, y finalmente agregamos un conmutador para enrutar mensajes entre varios clientes. La idea era ver, en la práctica, todo lo que implica mandar información por un medio: ponerse de acuerdo en un protocolo, lidiar con errores, y el costo de meter intermediarios en el camino.

3. Respuestas

3.1 ¿Qué esquema fue más fácil y cuál más difícil?

Morse fue el más fácil. Aunque hay que memorizar más símbolos, el punto y la raya suenan claramente distintos, así que uno los distingue sin mucho entrenamiento. Además el código se separa solo: los silencios entre letras y palabras te dicen dónde termina cada cosa, y podés ir segmentando el mensaje mientras lo escuchás.

Baudot fue el más difícil, y el problema principal fue reconocer los ceros. Al ser binario, hay que distinguir entre 1 y 0, pero identificar cuándo se estaba emitiendo un 0 se volvió complicadísimo. En Morse la ausencia de sonido es un separador; en Baudot el 0 es un símbolo que sí significa algo y hay que percibirlo activamente. Varias veces el receptor no sabía si había pasado un 0 o si solo era una pausa entre bits, y eso desalineaba el conteo por completo.

A eso se suma que Baudot obliga a llevar la cuenta exacta de 5 bits por carácter. Un bit perdido no arruina solo esa letra: corre la ventana de lectura y daña todo lo que viene después, así que tocaba reiniciar el mensaje. En Morse un error se queda en esa letra y en la siguiente ya te recuperaste.

3.2 ¿Qué esquema tuvo menos errores?

Morse, por bastante margen. Estos son los datos que recopilamos:

Esquema	Mensajes	Caracteres totales	Caracteres con error	Tasa de error
Morse	6	[]	[]	[] %
Baudot	6	[]	[]	[] %

Casi todos los errores de Baudot vienen de lo mismo: ceros mal interpretados y pérdida del conteo de bits. Los pocos errores en Morse fueron confusiones entre letras parecidas o por la velocidad del emisor, pero eran errores aislados que no arrastraban el resto del mensaje.



Figura 1: Registro de transmisión de la estación de telégrafo para uno de los mensajes en Morse.

3.3 ¿Qué dificultades involucra enviar un mensaje "empaquetado"?

Lo más notorio es que se pierde la retroalimentación en tiempo real. En vivo el receptor te interrumpe y te pide que repitas; con una nota de voz el mensaje sale completo y cerrado, sin forma de corregir sobre la marcha.

Eso obliga a retransmitir todo. Si al receptor se le escapa un solo carácter, no hay manera de pedir solo ese pedazo: hay que grabar la nota otra vez de principio a fin. Un bit ambiguo puede invalidar un paquete de más de un minuto.

Tampoco hay control sobre el ritmo. En persona uno baja la velocidad al ver que el otro se está quedando atrás; en una grabación la velocidad queda fija y al receptor le toca pausar y retroceder el audio, lo que genera errores nuevos por andar manipulando el reproductor.

Además hay que marcar explícitamente dónde empieza y termina el mensaje, porque sin el emisor presente no hay ninguna señal que lo indique. Y el medio degrada la señal: la compresión de audio de WhatsApp y el ruido de fondo hacen que los pulsos se escuchen menos limpios que en vivo.



Figura 2: Notas de voz intercambiadas durante la transmisión empaquetada (1:22 y 0:51 de duración por paquete).

3.4 ¿Qué posibilidades incluye introducir un conmutador?

Cambia bastante el sistema. Sin conmutador, cada par de personas necesita su propio enlace; con él, basta que cada cliente tenga una conexión al conmutador para poder alcanzar a cualquier otro. La cantidad de enlaces baja muchísimo.

También obliga a direccionar de forma explícita: como el mensaje puede ir a varios lados, hay que decir de quién viene y para quién es. De ahí salen los encabezados en las redes reales.

Y abre otras cosas: el conmutador puede guardar un mensaje si el destinatario no está disponible (store-and-forward), encolar el tráfico cuando varios transmiten a la vez, y decidir qué hacer con un paquete corrupto (retransmitirlo, descartarlo, priorizar otro).

3.5 ¿Ventajas y desventajas de agregar más conmutadores?

Ventajas

- Escala mejor: se pueden agregar más clientes sin saturar un solo nodo.
- Redundancia: si un conmutador cae, el tráfico puede irse por otra ruta.
- Menos cuellos de botella, porque la carga se reparte.
- Más alcance: permite conectar grupos que un solo nodo no cubriría.

Desventajas

- Más latencia: cada salto implica recibir, interpretar y reenviar.
- Los errores se acumulan: si un conmutador intermedio se equivoca, el error se propaga.
- Hay que decidir por dónde mandar cada mensaje, y eso puede generar bucles o duplicados.
- Más costo y más coordinación: todos deben compartir el mismo protocolo.

4. Protocolo que usamos

Cada nota de voz enviada al conmutador tenía tres partes. Al inicio decíamos en voz normal quién enviaba y a quién iba dirigido ("De Cliente 1 para Cliente 3") — ese era el encabezado. Luego venía el mensaje codificado, y al final un delimitador acordado para marcar el fin, para que el receptor no confundiera el silencio o el ruido final con parte del contenido.

El conmutador solo escuchaba el encabezado y reenviaba el paquete completo sin decodificarlo. Lo hicimos así a propósito: si el conmutador intentaba interpretar el contenido, podía meter errores que no le tocaban, y además tardaba más.

Para no saturarlo, antes de transmitir cada cliente pedía permiso con un mensaje corto. El conmutador respondía "listo" solo si no estaba procesando otro paquete; si no, tocaba esperar. Cuando el destinatario recibía el mensaje mandaba un ACK, y hasta entonces el conmutador se declaraba libre otra vez. Así solo se manejaba un mensaje a la vez.

Si el destinatario no lograba decodificar algo, pedía retransmisión al conmutador y este se lo pasaba al emisor original, que tenía que grabar la nota completa de nuevo. Lento, pero era la única opción sin retransmisión parcial.

5. Conclusiones

- Morse resultó más robusto que Baudot para transmisión humana, porque un error se queda contenido en una letra en lugar de arruinar todo el mensaje.
- La dificultad para reconocer los ceros muestra que un código no solo debe ser eficiente en bits: sus símbolos tienen que ser distinguibles en el medio real. Un símbolo que se confunde con la ausencia de señal está condenado a fallar.
- Mandar mensajes empaquetados dejó claro el costo de retransmitir todo por un solo error. De ahí la utilidad de la fragmentación, los checksums y los ACK en redes reales.
- El conmutador reduce enlaces y permite direccionar, pero sin control de flujo se vuelve rápidamente un cuello de botella y un punto único de falla.
- Más conmutadores dan escalabilidad y tolerancia a fallos, pero cuestan latencia y complejidad de enrutamiento — el mismo tradeoff de las redes reales.

- Al final, la complejidad de las redes no viene tanto del medio físico como de tener que acordar reglas comunes para entenderse, sincronizarse y recuperarse de los errores.

6. Referencias

Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Prentice Hall.

Wikipedia. Morse code. https://en.wikipedia.org/wiki/Morse_code

Wikipedia. Baudot code. https://en.wikipedia.org/wiki/Baudot_code